WEEKLY MINI PROJECT 03 · TEAM 4

Who Doesn't Know?

Work Package · Frontend C

Frontend | React + TS

PROJECT	TEAM LEADER	OWNER
Who Doesn't Know?	**Taima Watfeh**	**Alaa Ismaiel**

ROLE	AREA
Frontend developer	**Round Decision screen · Result screen**

> *Two focused screens: the moment the group decides, and the big reveal.*

1 Your Work at a Glance

#	Task	Main file(s)
T1	Your API file: decision	`api/decision.api.ts`
T2	Your mock data	`mocks/decision.mock.ts`, `mocks/result.mock.ts`
T3	Round Decision screen	`screens/RoundDecisionScreen.tsx`
T4	Result screen	`screens/ResultScreen.tsx`
T5	Your preview page	`dev/AlaaPreview.tsx`

2 Mock Data First

PHASE 1 · DAYS 1–3

Mocks ON

`VITE_USE_MOCKS=true`. Your api file returns data from **your own** mock file.

PHASE 2 · DAY 4

Mocks OFF

`VITE_USE_MOCKS=false`. The same api functions call the real backend.

PHASE 3 · DAY 4–5

Fix together

Run the flows on the real backend; fix any difference in your own files.

3 Your Tasks, Step by Step

T1 Your API file: decision

`src/api/decision.api.ts`

1. `submitDecision(session, choice: 'continue' | 'vote')` → `POST /games/:roomCode/decisions` with `{ playerId, choice }`.
2. `if (USE_MOCKS) return mock.submitDecision(choice)`, else `request(...)`.

```
import { request, USE_MOCKS } from './http';
import * as mock from '../mocks/decision.mock';
import type { Session } from '../types';

export function submitDecision(s: Session, choice: 'continue' | 'vote') {
  if (USE_MOCKS) return mock.submitDecision(choice);
  return request<{ ok: true }>('POST', `/games/${s.roomCode}/decisions`, { playerId: s.playerId,
choice });
}
```

✔ Done when: the screen works with mocks on and off.

T2 Your mock data

`src/mocks/decision.mock.ts` · `src/mocks/result.mock.ts`

1. `decision.mock.ts`: fixtures `decisionOpen` (2 / 5 chosen, `me.hasDecided false`), `decisionChosen` (`hasDecided true`), `decisionAfterTie` (with `lastOutcome` 2 continue · 2 vote).
2. `submitDecision`: returns `mockDelay({ ok: true })`; a second call rejects with "You already made your choice".
3. `result.mock.ts`: fixtures `informedWin` and `hiddenWin` with `result` filled and vote counts for 5 players.

✔ Done when: every situation of your two screens has a fixture.

T3 Round Decision screen

`src/pages/RoundDecisionPage.tsx`

1. Title: "Round {state.roundNumber} complete. Continue asking or vote?"
2. Two big buttons: **Continue asking** / **Vote** → `submitDecision`, then `refresh()`.
3. If `me.hasDecided`: disable both, show "Waiting for others...".
4. Show "{decision.submittedCount} / {decision.totalPlayers} have chosen". Never who chose what.
5. If `decision.lastOutcome` exists, a small line: "Last time: 3 Continue · 2 Vote".
6. Errors shown under the buttons.

✔ Done when: all decision fixtures render correctly, one choice only.

T4 Result screen

`src/pages/ResultPage.tsx`

1. Headline from `result.winner`: "The informed players win!" or "The hidden player wins!".
2. Reveal: "{hidden player name} was the hidden player" and "The secret was: {secretTip}".
3. Table: player name · votes, sorted by votes, accused player highlighted.

✔ Done when: both result fixtures show the correct winner, hidden player, secret and counts.

T5 **Your preview page**

`src/pages/AlaaPreviewPage.tsx`

1. Buttons for each fixture; render your screen with it, a fake `session` and `refresh = () => {}`.

✔ Done when: you can demo both screens without the backend or anyone else's code.

4 Fields You Use

```
state.roundNumber
state.players              // { ref, displayName, isHost }[]  -> map refs to names
state.decision             // { submittedCount, totalPlayers, lastOutcome }
state.result               // { winner, hiddenPlayerRef, secretTip, accusedRef, voteCounts }
me.hasDecided
```

```
// frontend/src/types.ts  (Day-0 foundation)
export interface Session { roomCode: string; playerId: string; playerRef: number; }
export interface ScreenProps {
  state: PublicGameState;      // from Yazan's polling hook
  me: PrivateRoleView;         // from Yazan's polling hook
  session: Session;            // send session to YOUR OWN api functions
  refresh: () => void;         // call after your action succeeds, to update immediately
}
```

5 Rules for You

- Only edit your own files listed above.
- Show only what the backend calculated: never compute the winner yourself.
- Mocks must match `types.ts` field by field.

6 Presentation Part

Your part in the English presentation: **the round decision and the result reveal during the demo**. Prepare 1–2 minutes and be ready to explain your own code.